МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для хранения, систематизации и публикации учебных материалов»

Выполнил: Николаев Алексей Владимирович

Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /
«___» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Николаеву Алексею Владимировичу

обучающемуся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для электронного архива учебных материалов с ролевым доступом»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати, документация по средствам разработки веб-приложений.

2. Содержание задания по курсовой работе (проекту) - перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Раздел 1. Анализ предметной области			
Задача 1.1. Анализ существующих веб-приложений для хранения, систематизации и публикации учебных материалов	10	06.04.2026	
Задача 1.2. Анализ программных средств разработки и выбор стека технологий	7	12.04.2026	
Задача 1.3. Формулировка цели, задач и требований к системе	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			

Разрабатываемый вопрос	**Объем, %**	**Срок**	**Примечание**
Задача 2.1. Определение ролей пользователей и прав доступа (администратор, преподаватель, студент)	6	25.04.2026	обязательный пункт
Задача 2.2. Проектирование функциональности приложения и сценариев использования	7	03.05.2026	
Задача 2.3. Проектирование модели данных для сущностей «Пользователь», «Категория», «Материал» и «Файл»	9	10.05.2026	не менее 3 сущностей
Задача 2.4. Проектирование структуры интерфейса и макетов основных страниц	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и шаблонов страниц	8	20.05.2026	
Задача 3.2. Реализация аутентификации и авторизации пользователей	7	24.05.2026	обязательный пункт
Задача 3.3. Реализация системы проверки прав доступа в соответствии с ролью пользователя	7	28.05.2026	обязательный пункт
Задача 3.4. Реализация CRUD-интерфейса для пользователей, категорий и материалов	10	01.06.2026	обязательный пункт
Задача 3.5. Реализация загрузки, скачивания и удаления файлов учебных материалов	8	04.06.2026	обязательный пункт
Задача 3.6. Реализация агрегирования данных и формирования статистики по материалам	6	06.06.2026	количество файлов, материалов по категориям и авторам
Раздел 4. Оформление итогов работы			
Задача 4.1. Тестирование основных пользовательских сценариев и проверка прав доступа	4	09.06.2026	
Задача 4.2. Подготовка Git-репозитория и итоговой версии исходного кода проекта	3	11.06.2026	
Задача 4.3. Подготовка демонстрационной версии приложения и материалов к защите	3	12.06.2026	
Задача 4.4. Оформление пояснительной записки в соответствии с требованиями	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«___» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания 02.04.2026 г.

Дата сдачи выполненной работы «___» _____ 2026 г.

Задание принял к исполнению

02.04.2026 г. _____ Николаев А. В.
(подпись)

Содержание

ВВЕДЕНИЕ 6
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ 7
1.1 Анализ существующих веб-приложений для хранения, систематизации и публикации учебных материалов . . . 7
1.2 Анализ программных средств разработки и выбор стека технологий 9
1.3 Формулировка цели, задач и требований к системе . . . 12
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ 16
2.1 Определение ролей пользователей и прав доступа . . . 16
2.2 Проектирование функциональности приложения и сценариев использования 17
2.3 Проектирование модели данных для сущностей 19
2.4 Проектирование структуры интерфейса и макетов основных страниц 21
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ 23
3.1 Разработка базовой структуры приложения и шаблонов страниц 23
3.2 Реализация аутентификации и авторизации пользователей 25
3.3 Реализация системы проверки прав доступа в соответствии с ролью пользователя 26
3.4 Реализация CRUD-интерфейса для пользователей, категорий и материалов 28
3.5 Реализация загрузки, скачивания и удаления файлов учебных материалов 29
3.6 Реализация агрегирования данных и формирования статистики по материалам 30
3.7 Тестирование основных пользовательских сценариев и проверка прав доступа 31
ЗАКЛЮЧЕНИЕ 38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 39
ПРИЛОЖЕНИЕ А 40

ВВЕДЕНИЕ

В условиях активного использования электронных образовательных ресурсов задача хранения, систематизации и предоставления учебных материалов приобретает особую актуальность. В учебном процессе преподаватели и студенты постоянно работают с лекциями, методическими указаниями, лабораторными заданиями, презентациями и иными файлами, которые должны быть доступны в удобной, структурированной и безопасной форме. При отсутствии единого веб-инструмента материалы нередко хранятся разрозненно: в облачных сервисах, мессенджерах, на личных страницах преподавателей или в локальных каталогах. Это затрудняет поиск нужной информации, усложняет контроль актуальности файлов и не позволяет полноценно разграничивать доступ пользователей.

Целью курсового проекта является разработка веб-приложения для хранения, систематизации и публикации учебных материалов с учетом роли пользователя. Для достижения данной цели в работе были решены задачи анализа предметной области, выбора стека технологий, проектирования структуры приложения и модели данных, а также реализации механизмов аутентификации, авторизации, проверки прав доступа, CRUD-операций, загрузки и скачивания файлов и формирования статистики.

Объектом исследования является процесс электронного хранения и предоставления учебных материалов в рамках образовательной деятельности. Предметом исследования выступают методы и средства проектирования и разработки веб-приложения, обеспечивающего централизованную работу с такими материалами. Практическая значимость проекта заключается в создании работоспособного программного решения, которое может использоваться как основа для учебного архива материалов кафедры, дисциплины или отдельного курса.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

В данной главе выполняется анализ предметной области, связанной с хранением, систематизацией и публикацией учебных материалов в электронном виде. Рассматриваются существующие подходы и программные средства, формулируются цель, задачи и основные требования к разрабатываемому веб-приложению.

1.1 Анализ существующих веб-приложений для хранения, систематизации и публикации учебных материалов

В современных образовательных организациях учебные материалы представлены преимущественно в электронном виде. К таким материалам относятся лекции, лабораторные работы, методические указания, практические задания, презентации, шаблоны отчетов и иные вспомогательные документы. Все перечисленные данные должны быть доступны преподавателям и студентам в удобной и структурированной форме, а также поддерживаться в актуальном состоянии в течение учебного процесса.

На практике хранение учебных материалов часто организовано нецентрализованно. Документы размещаются в различных облачных сервисах, мессенджерах, на личных страницах преподавателей или в файловых каталогах. Подобный подход приводит к нескольким типичным проблемам. Во-первых, усложняется поиск нужного материала, так как пользователь вынужден обращаться к нескольким источникам. Во-вторых, отсутствует единая структура хранения, что затрудняет систематизацию материалов по дисциплинам, темам, видам занятий или авторам. В-третьих, при отсутствии ролевого разграничения доступа возрастает риск несанкционированного изменения или удаления данных. Наконец, становится затруднительным контроль актуальности материалов и учет действий пользователей.

Существующие программные решения частично закрывают обозначенные потребности. К наиболее распространенным подходам относятся облачные файловые хранилища, системы дистанционного обучения и внутренние корпоративные порталы кафедр и учебных подразделений.

Тип решения	**Преимущества**	**Недостатки**
Облачные файловые хранилища	Простая загрузка и скачивание файлов, совместный доступ, привычный интерфейс	Слабая предметно-ориентированная структура, ограниченные возможности ролевого разграничения и статистики
Системы дистанционного обучения	Поддержка учебного процесса, разграничение доступа, публикация материалов и заданий	Избыточность функциональности для узкой задачи электронного архива, более сложное сопровождение
Внутренние сайты и порталы кафедр	Возможность адаптации под локальные потребности, относительная простота публикации	Отсутствие единого подхода к моделированию данных, слабая поддержка CRUD-операций и аналитики

Облачные сервисы, такие как Google Drive и аналогичные решения, удобны для обмена файлами, однако ориентированы прежде всего на файловое хранение. В них отсутству-

ет выраженная модель предметной области, которая необходима для учебного архива: категории материалов, авторство, ролевой доступ, фильтрация и агрегирование данных.

Системы дистанционного обучения, например Moodle и аналогичные платформы, предоставляют широкие возможности для организации учебного процесса. Они позволяют публиковать учебные материалы, управлять доступом и вести учет активности. Однако подобные решения во многих случаях являются слишком сложными для задач компактного архива учебных материалов. Для учебного проекта их архитектура и функциональная насыщенность могут быть избыточными.

Внутренние сайты кафедр и учебных подразделений обычно создаются под локальные задачи. Такие решения могут быть удобны в использовании, но часто развиваются бессистемно. В результате возникают трудности при расширении структуры данных, организации прав доступа и поддержке целостности информации.

Таким образом, проведенный анализ показывает целесообразность разработки отдельного веб-приложения, ориентированного именно на хранение, систематизацию и публикацию учебных материалов. Такое приложение должно быть проще полнофункциональных образовательных платформ, но при этом содержать обязательные механизмы аутентификации, авторизации, ролевого разграничения доступа, CRUD-операций, загрузки и скачивания файлов, а также базовой статистической обработки данных.

1.2 Анализ программных средств разработки и выбор стека технологий

Для реализации веб-приложения необходимо выбрать программные средства, позволяющие создать серверную логику, пользовательский интерфейс, механизм аутентификации, доступ к базе данных и обработку файлов. Поскольку

проект выполняется в учебных целях, основными критериями выбора являются простота освоения, доступность документации, достаточность функциональности и удобство поэтапной разработки.

В качестве языка программирования выбран Python. Данный язык обладает простым и читаемым синтаксисом, широко используется при разработке веб-приложений и имеет большой набор библиотек для решения прикладных задач. В качестве альтернативы можно рассматривать Rust, который обеспечивает высокую производительность и безопасность памяти, однако его использование требует более глубокого понимания низкоуровневых механизмов и усложняет разработку учебного проекта. Поэтому для данной работы выбран Python, так как он позволяет быстрее реализовать основную логику приложения и сосредоточиться на предметной области.

В качестве веб-фреймворка выбран Flask. Он представляет собой легковесное средство разработки веб-приложений, позволяющее реализовать маршрутизацию, шаблонизацию страниц, обработку форм, работу с файлами и интеграцию со сторонними библиотеками. В качестве альтернативы можно использовать Django, который предоставляет больше встроенных возможностей, включая административную панель, ORM и готовую структуру проекта. Однако для небольшого учебного приложения Django может быть избыточным и требовать большего объема настройки. Flask выбран потому, что он проще по структуре и удобен для поэтапной реализации необходимых функций.

Для аутентификации пользователей и управления сессиями выбран модуль Flask-Login. Он предоставляет готовые механизмы входа в систему, выхода из нее, восстановления текущего пользователя и ограничения доступа к маршрутам для неавторизованных пользователей. В качестве альтернативы можно было реализовать механизм аутентификации са-

мостоятельно с использованием сессий Flask. Однако самостоятельная реализация потребовала бы больше кода и увеличила бы вероятность ошибок при проверке авторизации. Flask-Login выбран потому, что он упрощает реализацию входа пользователей и делает работу с текущим пользователем более удобной и надежной.

Для работы с базой данных используется Flask-SQLAlchemy. Библиотека обеспечивает объектно-реляционное отображение, позволяя описывать сущности в виде Python-классов и выполнять операции с данными на уровне моделей. В качестве альтернативы можно использовать прямые SQL-запросы через модуль sqlite3. Такой подход дает больше контроля над запросами, однако усложняет поддержку кода и требует вручную описывать многие операции с данными. Flask-SQLAlchemy выбран потому, что он упрощает реализацию CRUD-интерфейса, уменьшает количество повторяющегося кода и делает структуру приложения более понятной.

В качестве системы управления базой данных выбрана SQLite. Ее преимуществами являются простота настройки, отсутствие необходимости устанавливать отдельный сервер базы данных и достаточность возможностей для учебного проекта. В качестве альтернативы можно использовать PostgreSQL, который лучше подходит для крупных и нагруженных приложений, поддерживает расширенные возможности и многопользовательскую работу. Однако для учебного проекта использование PostgreSQL усложнило бы настройку и развертывание. SQLite выбрана потому, что она не требует отдельного сервера и позволяет быстро приступить к разработке приложения.

Для построения пользовательского интерфейса используются HTML, CSS, Jinja2 и Bootstrap. HTML и CSS применяются для описания структуры и внешнего вида страниц, Jinja2 позволяет формировать динамические страницы

на стороне сервера, а Bootstrap обеспечивает единый и аккуратный внешний вид интерфейса. В качестве альтернативы можно использовать отдельный JavaScript-фреймворк, например React. Такой подход удобен для сложных интерактивных интерфейсов, однако требует разработки отдельной клиентской части и дополнительной настройки взаимодействия с сервером. Для данного проекта выбран серверный рендеринг с Jinja2 и Bootstrap, так как он проще в реализации и полностью соответствует требованиям учебного веб-приложения.

Компонент	**Выбранное средство**	**Причина выбора**
Язык программирования	Python	Простота синтаксиса, высокая скорость разработки, хорошая документация
Веб-фреймворк	Flask	Легковесность, гибкость, удобство для учебного проекта
Аутентификация	Flask-Login	Готовые механизмы входа, выхода и работы с пользовательской сессией
Доступ к БД	Flask-SQLAlchemy	Удобная работа с моделями, ORM-подход, упрощение CRUD-операций
СУБД	SQLite	Простая настройка, отсутствие отдельного серверного ПО, достаточность для проекта
Шаблоны и интерфейс	Jinja2, Bootstrap	Динамическая генерация страниц и единый визуальный стиль
Работа с файлами	Средства Flask и Werkzeug	Простая загрузка, сохранение и скачивание файлов

Таким образом, выбранный стек технологий включает Python, Flask, Flask-Login, Flask-SQLAlchemy, SQLite, Jinja2 и Bootstrap.

1.3 Формулировка цели, задач и требований к системе

Целью курсового проекта является разработка веб-приложения для хранения, систематизации и публикации учебных материалов с разграничением прав доступа пользователей в зависимости от их роли.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область хранения и предоставления учебных материалов;
- определить категории пользователей системы и их права доступа;
- выделить основные сущности предметной области и связи между ними;
- спроектировать структуру базы данных и интерфейс веб-приложения;
- реализовать аутентификацию и авторизацию пользователей;
- реализовать систему проверки прав доступа в соответствии с ролью пользователя;
- реализовать CRUD-интерфейс для пользователей, категорий и материалов;
- реализовать загрузку, скачивание и удаление файлов учебных материалов;
- реализовать агрегирование данных и формирование статистики;
- выполнить тестирование и подготовить пояснительную документацию.

В рамках системы выделяются следующие категории пользователей:

Роль	Основные функции
Администратор	Управление пользователями, категориями и всеми материалами, просмотр статистики, полный контроль над системой
Преподаватель	Создание и редактирование собственных материалов, загрузка и удаление файлов, просмотр категорий и материалов
Студент	Просмотр списка материалов, открытие карточек материалов, скачивание прикрепленных файлов

В приложении представлены следующие сущности:

Сущность	Назначение
Пользователь	Хранение сведений об учетной записи, логине, пароле, ФИО и роли
Роль	Определение уровня прав доступа пользователя
Категория	Классификация учебных материалов по тематике или типу
Материал	Основная сущность архива, содержащая название, описание, автора и категорию
Файл	Сведения о прикрепленном файле учебного материала и его хранении

К основным функциональным требованиям относятся:

- регистрация и аутентификация пользователей администраторами системы;
- авторизация и разграничение прав доступа по ролям;
- создание, просмотр, редактирование и удаление пользователей;
- создание, просмотр, редактирование и удаление категорий;

- создание, просмотр, редактирование и удаление материалов;
- загрузка, скачивание и удаление файлов, связанных с материалами;
- фильтрация и поиск материалов;
- формирование агрегированных данных и статистики.

К основным нефункциональным требованиям относятся удобство интерфейса, корректная работа в современных браузерах, поддержание целостности данных, ограничение доступа в соответствии с ролями и возможность дальнейшего расширения системы.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

На данном этапе определяются роли пользователей и их права доступа, проектируются основные сценарии использования системы, формируется модель данных и описывается структура интерфейса. Результаты проектирования служат основой для последующей программной реализации системы.

2.1 Определение ролей пользователей и прав доступа

Для обеспечения безопасной и логичной работы системы необходимо заранее определить категории пользователей и круг доступных им действий. В проектируемом приложении используются три роли: администратор, преподаватель и студент.

Администратор обладает максимальным набором прав и отвечает за общее сопровождение системы. Он управляет учетными записями пользователей, ролями, категориями и всеми материалами, а также имеет доступ к просмотру статистики. Преподаватель работает с учебными материалами: создает их, редактирует собственные записи, прикрепляет файлы и при необходимости удаляет их. Студент использует приложение только как средство получения учебной информации, поэтому для него доступен просмотр материалов и скачивание файлов.

Матрица прав доступа представлена в таблице.

Функция	Адм.	Препод.	Студ.
Вход в систему и выход из нее	+	+	+
Просмотр списка материалов	+	+	+
Просмотр карточки материала	+	+	+
Создание материалов	+	+	–
Редактирование собственных материалов	+	+	–
Удаление материалов	+	+	–
Загрузка файлов к материалам	+	+	–
Удаление файлов из материалов	+	+	–
Просмотр категорий	+	+	–
Управление категориями	+	–	–
Управление пользователями	+	–	–
Просмотр статистики	+	–	–

Выбранное распределение прав позволяет сохранить необходимый уровень безопасности и при этом не усложняет взаимодействие пользователей с системой.

2.2 Проектирование функциональности приложения и сценариев использования

Проектируемое приложение должно обеспечивать базовый набор пользовательских сценариев, связанных с управлением учебными материалами. На этапе проектирования были выделены основные функции системы: вход пользователя в систему, управление учетными записями, управление категориями, работа с материалами и файлами, а также просмотр статистики.

Основные пользовательские сценарии можно описать следующим образом:

- администратор входит в систему, создает пользователей, назначает им роли, управляет категориями и контролирует наполнение архива;
- преподаватель входит в систему, создает новый матери-

ал, заполняет его описание, выбирает категорию и прикрепляет файлы;

- преподаватель при необходимости редактирует собственный материал, обновляет информацию и удаляет ненужные файлы;
- студент входит в систему, просматривает список материалов, открывает карточку нужного материала и скачивает прикрепленный файл;
- администратор открывает страницу статистики и анализирует распределение материалов по категориям, авторам и ролям пользователей.

Для наглядного представления взаимодействия пользователей с системой в проект была включена диаграмма вариантов использования. На диаграмме отражены основные акторы системы — администратор, преподаватель и студент — и показаны ключевые действия, которые доступны каждой роли.

Диаграмма вариантов использования подтверждает, что проектируемая система охватывает все основные пользовательские сценарии, предусмотренные заданием: аутентификацию, разграничение прав доступа, работу с материалами и файлами, а также просмотр статистики администратором.

Для удобства навигации проектируется следующий состав страниц приложения:

Страница	**Назначение**
Главная страница	Краткая информация о системе и переход к основным разделам
Страница входа	Ввод логина и пароля пользователя

Страница	Назначение
Список пользователей	Просмотр и управление учетными записями
Список категорий	Просмотр и администрирование категорий материалов
Список материалов	Просмотр всех материалов с возможностью фильтрации
Карточка материала	Просмотр описания материала и прикрепленных файлов
Страница создания и редактирования материала	Ввод названия, описания, категории и загрузка файлов
Страница статистики	Отображение агрегированных сведений по материалам, файлам и пользователям

Спроектированная функциональность приложения соответствует поставленной цели и позволяет реализовать все обязательные элементы курсового проекта.

2.3 Проектирование модели данных для сущностей

На этапе проектирования модели данных были выделены основные сущности предметной области: `Role`, `User`, `Category`, `Material` и `StoredFile`. Каждая из них отражает отдельный аспект работы системы.

Сущность `Role` используется для хранения названия роли и ее описания. Сущность `User` содержит данные пользователя: логин, хэш пароля, ФИО, дату создания учетной записи и ссылку на роль. Сущность `Category` предназначена для систематизации учебных материалов. Сущность `Material` хранит сведения о самом материале: название, описание, да-

Рисунок 2.1 – Диаграмма вариантов использования веб-приложения

ту создания, автора и категорию. Сущность `StoredFile` содержит информацию о прикрепленных файлах: оригинальное имя, внутреннее имя хранения, путь к файлу и дату загрузки.

Связи между сущностями можно описать следующим образом:

- одной роли соответствует множество пользователей;
- одному пользователю может соответствовать множество материалов;
- одной категории может соответствовать множество материалов;
- одному материалу может соответствовать множество прикрепленных файлов.

Проектируемая модель данных представлена в таблице.

Сущность	Основные атрибуты	Назначение
Role	id, name, description	Хранение ролей пользователей
User	id, username, password_hash, full_name, created_at, role_id	Хранение учетных записей пользователей
Category	id, name, description	Классификация учебных материалов
Material	id, title, description, created_at, author_id, category_id	Описание учебного материала
StoredFile	id, original_name, stored_name, file_path, uploaded_at, material_id	Хранение информации о прикрепленных файлах

Такая модель данных достаточно проста для реализации, но при этом полностью покрывает все требования.

2.4 Проектирование структуры интерфейса и макетов основных страниц

На этапе проектирования интерфейса была принята цель сделать приложение максимально простым и понятным. Пользователь должен быстро переходить между основными разделами системы и выполнять типовые операции без лишних действий.

В основе интерфейса лежит единый базовый шаблон страницы, содержащий верхнее меню навигации, название

системы, ссылки на доступные разделы и область отображения системных сообщений. Такое решение обеспечивает единый внешний вид всех страниц приложения.

Для основных страниц были определены следующие элементы интерфейса:

- для страницы входа — форма авторизации с полями логина и пароля;
- для страницы пользователей — таблица со списком учетных записей и кнопками создания, редактирования и удаления;
- для страницы категорий — таблица категорий и форма их добавления или изменения;
- для страницы материалов — таблица или карточки материалов, фильтр по категории и ссылки на просмотр и редактирование;
- для карточки материала — название, описание, сведения об авторе, категория, список прикрепленных файлов и кнопки действий;
- для страницы статистики — набор таблиц с агрегированными данными по материалам, файлам, категориям и пользователям.

Проектируемая структура интерфейса позволяет легко масштабировать приложение и при необходимости дополнять его новыми разделами. При этом выбранный подход не перегружает пользователя и соответствует учебному характеру проекта.

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

Разработанная система ориентирована на хранение, систематизацию и публикацию учебных материалов с учетом роли пользователя. В ходе разработки были реализованы базовая структура проекта, аутентификация, авторизация, механизм проверки прав доступа, CRUD-интерфейс для основных сущностей, загрузка и скачивание файлов, а также формирование статистики по материалам.

3.1 Разработка базовой структуры приложения и шаблонов страниц

Была сформирована базовая структура проекта, обеспечивающая разделение логики приложения, описания моделей данных, маршрутов, шаблонов страниц и конфигурации. Такой подход упрощает сопровождение программного кода и делает проект более понятным.

Структура приложения представлена следующими основными компонентами:

- файл `run.py` — точка запуска веб-приложения;
- файл `config.py` — конфигурация системы, включая путь к базе данных, папке загрузок и список допустимых расширений файлов;
- модуль `app/__init__.py` — инициализация Flask-приложения, объекта базы данных, менеджера авторизации и вспомогательных шаблонных фильтров;
- модуль `app/models.py` — описание сущностей предметной области и связей между ними;
- модуль `app/routes.py` — обработка HTTP-запросов и реализация пользовательских сценариев;
- модуль `app/decorators.py` — декораторы для проверки ролей пользователей;

- каталог `app/templates/` — HTML-шаблоны страниц;
- каталог `app/uploads/` — сохранение загруженных учебных файлов.

В файле конфигурации были заданы ключевые параметры приложения: секретный ключ, адрес базы данных SQLite, путь к каталогу загрузки файлов, максимальный размер загружаемого файла и набор разрешенных расширений. Это обеспечивает централизованное управление настройками и упрощает перенос приложения в другую среду.

Базовые шаблоны страниц построены по единому принципу: каждая страница наследуется от общего базового шаблона, содержащего навигационное меню, блок вывода сообщений системы и общую структуру пользовательского интерфейса. В результате был сформирован единый стиль страниц и обеспечено удобство навигации по разделам приложения.

В интерфейсе реализованы следующие основные страницы:

Страница	Назначение	Кому доступна
Главная страница	Краткая информация о системе и последние добавленные материалы	Всем авторизованным пользователям
Страница входа	Ввод логина и пароля для начала работы в системе	Неавторизованным пользователям
Пользователи	Просмотр, создание, редактирование и удаление учетных записей	Администратору
Категории	Просмотр и управление категориями учебных материалов	Администратору и преподавателю
Материалы	Просмотр списка материалов, фильтрация и переход к карточке материала	Всем авторизованным пользователям
Карточка материала	Просмотр описания материала и прикрепленных файлов	Всем авторизованным пользователям
Статистика	Агрегированные сведения по материалам, пользователям и файлам	Администратору

Таким образом, была создана базовая структура Flask-приложения и набор шаблонов, необходимых для дальнейшей реализации предметной логики.

3.2 Реализация аутентификации и авторизации пользователей

Следующим этапом была реализована аутентификация пользователей. Для этого использована библиотека Flask-Login, позволяющая организовать сеансы работы пользователя, хранить состояние входа и ограничивать доступ к защищенным страницам.

В приложении реализована форма входа, содержащая поля логина и пароля. После отправки формы выполняется поиск пользователя по логину и проверка корректности пароля с использованием хэширования. В случае успешной про-

верки создается пользовательская сессия, после чего пользователь перенаправляется на главную страницу приложения. При ошибочном вводе данных выводится соответствующее сообщение.

Для хранения паролей используется не открытый текст, а хэш, вычисляемый средствами библиотеки Werkzeug. Это позволяет повысить безопасность приложения и соответствует типовой практике разработки веб-систем.

Ниже приведен фрагмент кода, реализующий проверку пароля в модели пользователя:

```
1 def set_password(self, password: str) -> None:
2     self.password_hash = generate_password_hash(password)
3
4 def check_password(self, password: str) -> bool:
5     return check_password_hash(self.password_hash, password)
```

Для работы Flask-Login в проекте настроен загрузчик пользователя по идентификатору, что обеспечивает восстановление сведений о текущем пользователе из пользовательской сессии:

```
1 @login_manager.user_loader
2 def load_user(user_id):
3     return User.query.get(int(user_id))
```

После успешного входа пользователь получает доступ только к тем страницам и действиям, которые разрешены его ролью. При попытке обращения к защищенным маршрутам без авторизации система выдает сообщение о необходимости входа в систему.

Таким образом, в приложении была реализована полноценная аутентификация пользователей и базовая авторизация на уровне пользовательской сессии.

3.3 Реализация системы проверки прав доступа в соответствии с ролью пользователя

В разработанном приложении реализованы три роли:

- `admin` — администратор;
- `teacher` — преподаватель;
- `student` — студент.

Роль пользователя хранится в отдельной сущности `Role`, а в сущности `User` предусмотрена связь с соответствующей записью роли. Для проверки ролей в модели пользователя реализован метод `has_role()`, возвращающий истинное значение, если роль пользователя входит в заданный набор.

Для ограничения доступа к маршрутам был создан собственный декоратор `roles_required`. Он используется совместно с декоратором `@login_required` и проверяет, обладает ли текущий пользователь допустимой ролью. Если пользователь не авторизован, возвращается код 401, а если его роль не соответствует требуемой, возвращается код 403.

```
1  def roles_required(*role_names):
2      def decorator(view_func):
3          @wraps(view_func)
4          def wrapped(*args, **kwargs):
5              if not current_user.is_authenticated:
6                  abort(401)
7              if not current_user.has_role(*role_names):
8                  abort(403)
9              return view_func(*args, **kwargs)
10         return wrapped
11     return decorator
```

Кроме проверки роли на уровне маршрута, для операций над материалами реализована дополнительная проверка принадлежности материала пользователю. Это необходимо для того, чтобы преподаватель мог редактировать и удалять только собственные материалы, тогда как администратор может управлять всеми материалами.

Следовательно, в приложении реализовано ролевое разграничение доступа, полностью соответствующее требованиям.

3.4 Реализация CRUD-интерфейса для пользователей, категорий и материалов

В приложении реализованы операции создания, просмотра, редактирования и удаления для трех основных сущностей: пользователей, категорий и материалов. Все операции доступны через веб-интерфейс и соответствующие маршруты Flask.

CRUD для пользователей

Управление пользователями доступно только администратору. Он может просматривать список учетных записей, создавать новых пользователей, изменять логин, ФИО, пароль и роль, а также удалять пользователей. Для защиты системы предусмотрено ограничение, запрещающее администратору удалить собственную учетную запись во время работы в системе.

CRUD для категорий

Категории используются для систематизации учебных материалов. Администратор может создавать новые категории, изменять их название и описание, а также удалять категории. При этом реализована проверка, запрещающая удаление категории, если с ней уже связаны материалы. Такое ограничение поддерживает целостность данных.

CRUD для материалов

Материал является основной сущностью приложения. При создании материала указываются его название, описание и категория. Автором материала автоматически становится текущий авторизованный пользователь. После создания материал отображается в общем списке и доступен для просмотра пользователям с соответствующими правами.

При редактировании материала допускается изменение названия, описания и категории, а также добавление новых файлов. В режиме редактирования отображается список уже загруженных файлов, для которых реализованы отдельные действия по скачиванию и удалению.

Основные CRUD-маршруты приложения представлены в таблице.

Маршрут	**Назначение**	**Методы**
/users	Список пользователей	GET
/users/create	Создание пользователя	GET, POST
/users/id/edit	Редактирование пользователя	GET, POST
/users/id/delete	Удаление пользователя	POST
/categories	Список категорий	GET
/categories/create	Создание категории	GET, POST
/categories/id/edit	Редактирование категории	GET, POST
/categories/id/delete	Удаление категории	POST
/materials	Список материалов и фильтрация	GET
/materials/create	Создание материала	GET, POST
/materials/id	Карточка материала	GET
/materials/id/edit	Редактирование материала	GET, POST
/materials/id/delete	Удаление материала	POST

Таким образом, в системе реализован полноценный CRUD-интерфейс для основных сущностей предметной области.

3.5 Реализация загрузки, скачивания и удаления файлов учебных материалов

Одной из ключевых функций приложения является работа с учебными файлами. Для этого в проекте реализована отдельная сущность `StoredFile`, содержащая оригинальное имя файла, внутреннее имя для хранения, путь к файлу на диске, дату загрузки и связь с материалом.

При загрузке файла выполняется проверка расширения. Разрешены только типы файлов, соответствующие учебным материалам: PDF, DOC, DOCX, TXT, PPT, PPTX, XLS, XLSX, PNG, JPG, JPEG. После проверки файл сохраняется в каталог `app/uploads`. Для предотвращения конфликтов имен и перезаписи существующих файлов используется генерация уникального имени на основе UUID.

Фрагмент кода сохранения файлов имеет следующий вид:

```
original_name = secure_filename(file.filename)
extension = original_name.rsplit('.', 1)[1].lower()
stored_name = f"{uuid.uuid4().hex}.{extension}"
file_path = os.path.join(upload_folder, stored_name)
file.save(file_path)
```

Скачивание файла реализовано отдельным маршрутом. При обращении к нему сервер находит файл по идентификатору и отправляет его пользователю как вложение, сохраняя исходное имя файла для скачивания.

Удаление файла также выполняется отдельным маршрутом. При этом сначала удаляется физический файл с диска, затем удаляется соответствующая запись из базы данных. Возможность удаления предоставляется только администратору или преподавателю, имеющему право управления данным материалом.

Реализованный механизм загрузки, скачивания и удаления файлов позволяет полностью закрыть одно из обязательных требований, связанное с обработкой файловых данных.

3.6 Реализация агрегирования данных и формирования статистики по материалам

Для выполнения требования, связанного с обработкой и агрегированием данных, в приложении была реализована отдельная страница статистики. Доступ к ней предоставляется только администратору.

На странице статистики формируются следующие агрегированные показатели:

- общее количество материалов в системе;
- общее количество загруженных файлов;
- количество материалов по категориям;
- количество материалов по авторам;
- количество пользователей по ролям.

Для формирования статистики используются агрегатные функции SQLAlchemy, в частности `COUNT()`, а также операции группировки по категориям, авторам и ролям. Пример формирования статистики по категориям представлен ниже:

```
materials_by_category = (
    db.session.query(Category.name, func.count(Material.id))
    .outerjoin(Material, Material.category_id == Category.id)
    .group_by(Category.id)
    .order_by(Category.name)
    .all()
)
```

Полученные значения передаются в шаблон страницы статистики и выводятся в виде наглядных таблиц. Это позволяет администратору оценивать наполнение архива, распределение материалов по категориям и активность пользователей.

3.7 Тестирование основных пользовательских сценариев и проверка прав доступа

После завершения реализации основных модулей приложения было выполнено функциональное тестирование пользовательских сценариев. Целью тестирования являлась проверка корректности работы разработанного веб-приложения в условиях, приближенных к его реальному использованию,

а также подтверждение выполнения требований курсового проекта, связанных с аутентификацией, авторизацией, разграничением прав доступа, CRUD-операциями, обработкой файлов и формированием статистики.

Тестирование проводилось методом последовательной ручной проверки ключевых пользовательских сценариев. В качестве тестовых ролей использовались три учетные записи: администратор, преподаватель и студент. Проверка включала как позитивные сценарии, при которых пользователь выполняет допустимые действия, так и негативные сценарии, при которых система должна запретить доступ к функции или странице. Основное внимание уделялось корректности маршрутизации, сохранению данных в базе, изменению интерфейса в зависимости от роли пользователя, а также работе операций загрузки, скачивания и удаления файлов.

Для проверки были подготовлены тестовые данные: несколько пользователей с различными ролями, набор категорий учебных материалов, несколько записей материалов и прикрепленные к ним файлы разных типов. После каждого действия проверялось не только визуальное изменение данных на странице, но и фактическое наличие или отсутствие соответствующих записей в базе данных и файловом хранилище приложения.

Результаты тестирования основных сценариев приведены ниже.

№	**Проверяемый сценарий**	**Ожидаемый результат**	**Фактический результат**
1	Открытие защищенной страницы без авторизации	Система перенаправляет пользователя на страницу входа	Выполнено, доступ без входа не предоставляется

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
2	Вход администратора по корректным учетным данным	Открывается главная страница приложения, пользователь-ская сессия создается	Выполнено, вход успешен
3	Выход из системы	Сессия пользователя завершается, защищенные страницы снова недоступны	Выполнено, повторный доступ требует авторизации
4	Создание нового пользователя администратором	В базе появляется новая учетная запись с выбранной ролью	Выполнено, пользователь создается и отображается в списке
5	Редактирование данных пользователя администратором	Измененные значения сохраняются и отображаются в интерфейсе	Выполнено, обновленные данные отображаются корректно
6	Удаление пользователя администратором	Учетная запись удаляется из базы и исчезает из списка	Выполнено, запись удаляется корректно

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
7	Создание категории администратором	Новая категория сохраняется и доступна при создании материалов	Выполнено, категория появляется в списке
8	Попытка студента открыть страницу категорий	Доступ запрещается, страница недоступна студенту	Выполнено, доступ студенту ограничен
9	Создание нового материала преподавателем	Материал сохраняется с указанием автора и категории	Выполнено, материал добавляется в систему
10	Редактирование материала его автором	Изменения сохраняются и отображаются на странице просмотра материала	Выполнено, материал обновляется корректно
11	Попытка студента изменить существующий материал	Система запрещает выполнение операции	Выполнено, права студента ограничены

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
12	Загрузка файла при создании или редактировании материала	Файл сохраняется на сервере и отображается в карточке материала	Выполнено, файл доступен в списке вложений
13	Скачивание прикрепленного файла авторизованным пользователем	Браузер начинает загрузку выбранного файла	Выполнено, файл скачивается корректно
14	Удаление файла из карточки материала пользователем с соответствующими правами	Файл удаляется из хранилища и перестает отображаться в интерфейсе	Выполнено, удаление работает корректно
15	Открытие страницы статистики администратором	Отображаются агрегирован-ные показатели по материалам, файлам и пользователям	Выполнено, статистика формируется корректно
16	Попытка преподавателя открыть страницу статистики	Доступ запрещается, страница статистики недоступна преподавателю	Выполнено, доступ ограничен

№	Проверяемый сценарий	Ожидаемый результат	Фактический результат
17	Попытка студента открыть страницу статистики	Доступ запрещается, страница статистики недоступна студенту	Выполнено, доступ ограничен
18	Проверка отображения автора материала в интерфейсе	Для каждого пользователя отображается формат “ФИО (логин)”	Выполнено, отображение авторов единообразно
19	Проверка отображения времени создания записей	Дата и время выводятся с учетом часового пояса МСК (UTC+3)	Выполнено, время отображается корректно
20	Проверка агрегированных данных после добавления новых материалов и файлов	Количественные показатели изменяются в соответствии с новыми данными	Выполнено, статистика обновляется корректно

По результатам проведенного тестирования можно сделать вывод, что основные пользовательские сценарии выполняются корректно. Проверка показала, что приложение устойчиво отрабатывает как обычные операции пользователя, так и ограничения, связанные с ролью учетной записи. Особенно важным результатом является подтверждение того, что студент не имеет доступа к административным функциям, преподаватель не может просматривать статистику, а

изменение и удаление материалов и файлов доступно только пользователям, обладающим соответствующими правами.

Также в процессе тестирования были выявлены и устранены отдельные недочеты интерфейса и серверной логики. К ним относились вопросы отображения времени, единообразного показа имени пользователя в формате “ФИО (логин)”, а также корректная обработка удаления уже загруженных файлов со страницы редактирования материала. После внесения исправлений указанные сценарии были протестированы повторно и подтвердили корректность реализации.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсового проекта было разработано веб-приложение для хранения, систематизации и публикации учебных материалов с ролевым разграничением доступа. В ходе работы была проанализирована предметная область, рассмотрены существующие подходы к организации электронных архивов учебных материалов, а также выбран технологический стек, включающий Python, Flask, Flask-Login, Flask-SQLAlchemy, SQLite, Jinja2 и Bootstrap.

На этапе проектирования были определены роли пользователей, основные сценарии использования системы, структура интерфейса и модель данных для сущностей, связанных с хранением материалов и файлов. На этапе разработки была реализована базовая структура приложения, аутентификация и авторизация пользователей, проверка прав доступа в соответствии с ролью, CRUD-интерфейс для пользователей, категорий и материалов, механизм загрузки, скачивания и удаления файлов, а также страница статистики с агрегированными данными.

Проведенное тестирование основных пользовательских сценариев подтвердило корректность работы реализованного приложения. Система обеспечивает доступ к функциям в соответствии с ролью пользователя, поддерживает работу с материалами и файлами и удовлетворяет обязательным требованиям курсового проекта. Полученный результат может рассматриваться как работоспособная основа для дальнейшего расширения системы, например для добавления более сложного поиска, журналирования действий пользователей, расширенной аналитики и интеграции с иными образовательными сервисами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Flask Documentation. — 2026. — URL: https://flask.palletsprojects.com/ (online; accessed: 2026-04-23).

[2] Flask-Login Documentation. — 2026. — URL: https://flask-login.readthedocs.io/ (online; accessed: 2026-04-23).

[3] SQLAlchemy Documentation. — 2026. — URL: https://docs.sqlalchemy.org/ (online; accessed: 2026-04-23).

[4] SQLite Documentation. — 2026. — URL: https://www.sqlite.org/docs.html (online; accessed: 2026-04-23).

[5] Bootstrap Documentation. — 2026. — URL: https://getbootstrap.com/docs/ (online; accessed: 2026-04-23).

ПРИЛОЖЕНИЕ А

В приложении приведена диаграмма вариантов использования разработанного веб-приложения, отражающая основные роли пользователей и ключевые сценарии взаимодействия с системой.

Рисунок А.1 – Диаграмма вариантов использования веб-приложения